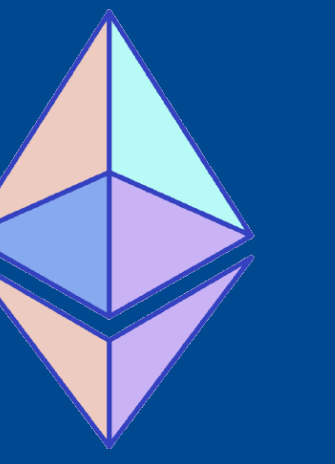




Formal Verification of Number-Theoretic Transform in Lean

Salih Erdem Koçak, Doran Pamukçu

Advisor: Doğan Ulus Department of Computer Engineering, Boğaziçi University

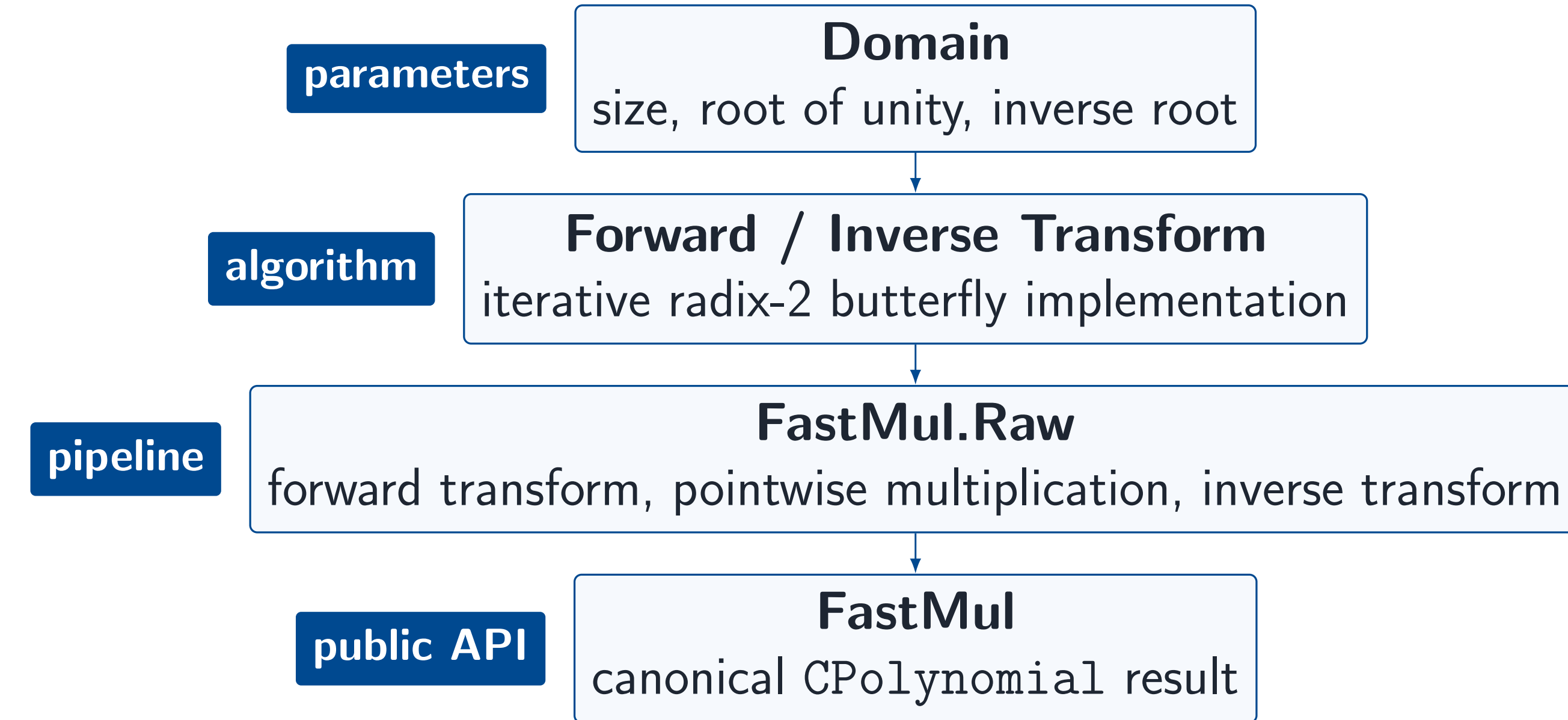


Motivation

Polynomial multiplication is a core component of many cryptographic protocols, especially proof systems. Evaluation and interpolation routines, polynomial commitments, and quotient-polynomial constructions all multiply polynomials repeatedly. With the usual schoolbook algorithm, this is quadratic work. The NTT avoids direct convolution by evaluating both inputs on a root-of-unity domain, multiplying the values pointwise, and then transforming back. Our Lean proof connects this executable pipeline to the algebraic specification, showing that the fast path is correct.

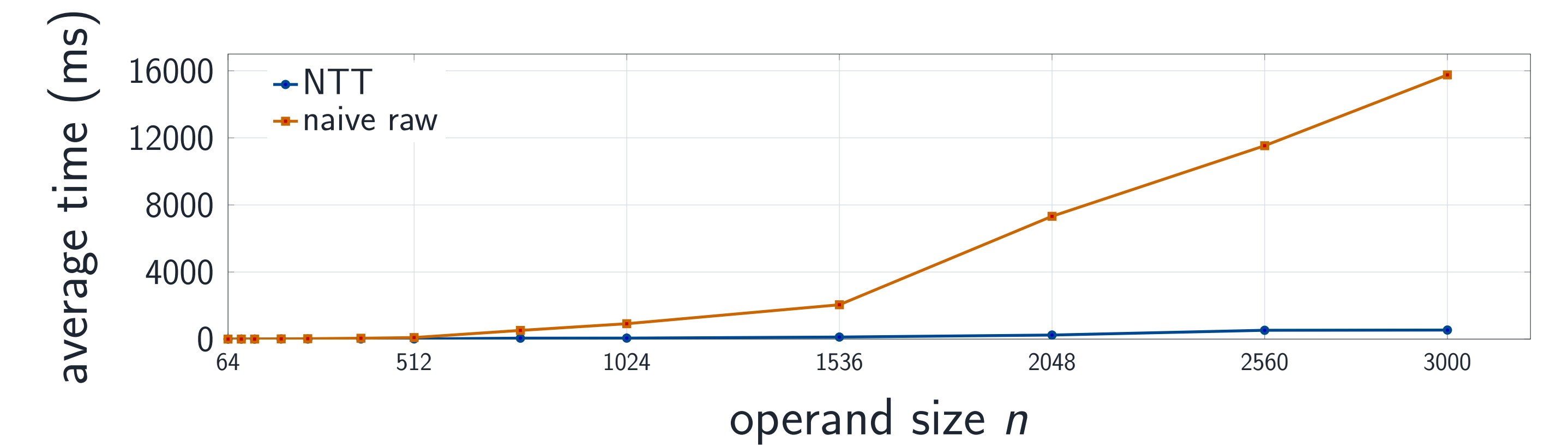
Method	Main Operation	Work
Naive	sum coefficient pairs	$O(n^2)$
NTT	transform, pointwise multiply, transform back	$O(n \log n)$

Implementation Surface



The public operation is small; most of the work is in connecting executable array code to mathematical transform specifications and then to polynomial multiplication.

Benchmark Behavior



Manual KoalaBear benchmark sweep from tests/CompPolyTests/Univariate/NTT/Benchmark.lean. Times are illustrative and machine-dependent.

Main Theorem

```

theorem fastMulImpl_eq_mul
  (D : Domain R)
  (p q : CPolynomial R)
  (hfit : Domain.fits D p.val q.val) :
  fastMulImpl D p q = p * q
  
```

This is the public correctness statement for the multiplication implementation. The precondition `Domain.fits D p.val q.val` says that the selected NTT domain is large enough for the ordinary convolution, so the transform pipeline has no cyclic wraparound.

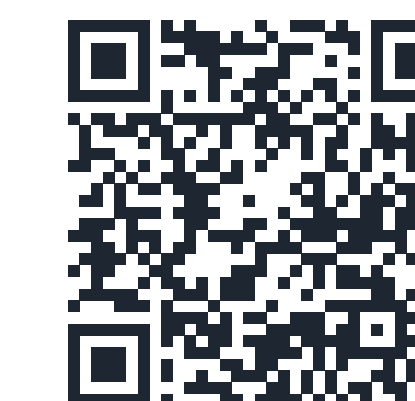
Proof Graph Guide

1. `fastMulImpl_eq_mul`: main theorem — the NTT-based implementation equals canonical polynomial multiplication.
2. `fastMulImpl_correct`: the executable multiplication pipeline agrees with the specification.
3. `fastMulSpec_eq_mul`: the specification computes the canonical polynomial product.
4. **Raw layer**: array-level correctness together with trimming and canonicalization lemmas.
5. **Forward / Inverse layer**: iterative radix-2 butterfly implementations realize the mathematical NTT and inverse NTT formulas.
6. **Coefficient layer**: coefficient-wise correctness follows from pointwise evaluation and inversion identities.
7. **Foundation layer**: inductive stage lemmas and root-of-unity orthogonality collapse the transform sums.

References

1. Trieu, A., “Formally Verified Number-Theoretic Transform.” *IACR Communications in Cryptology*, vol. 2, no. 4, Jan. 2026. DOI: 10.62056/ahbn-4tw9.
2. Capretta, V., “Certified Fast Fourier Transform.” *Theorem Proving in Higher Order Logics*, TPHOLs 2001. Springer, 2001.
3. Ballarin, C., “Fast Fourier Transform.” *Archive of Formal Proofs*, Oct. 2005. isa-afp.org/entries/FFT.html.
4. Truth Research ZK, “AMO-Lean”, 2026. github.com/lambdaclass/amo-lean.

Code and Repository

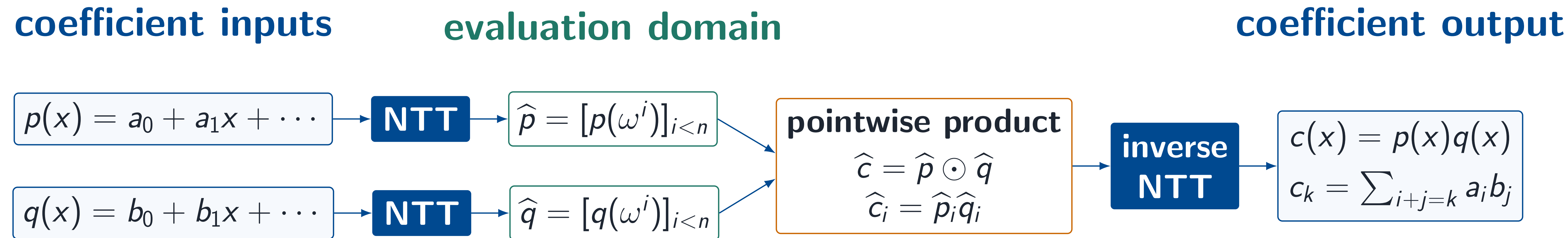


Merged PR #174



CompPoly Repository

NTT Multiplication



The transform evaluates at powers of a root of unity. Multiplication becomes pointwise in the evaluation domain, then the inverse transform returns the coefficient representation.

Proof Architecture

